下面是 **SENet、CBAM、ECA、CoordAttention（CA）** 四种注意力模块的**精炼 PyTorch 实现**（均可直接嵌入卷积块或残差块），以及**原始参考文献**。

---

1) SENet（Squeeze-and-Excitation）

```
import torch
import torch.nn as nn

class SEBlock(nn.Module):
    """
    SENet: Channel attention via GAP -> MLP(reduce/expand) -> Sigmoid
    """
    def __init__(self, channels: int, reduction: int = 16):
        super().__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)  # (B,C,H,W)->(B,C,1,1)
        self.fc = nn.Sequential(
            nn.Linear(channels, channels // reduction, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(channels // reduction, channels, bias=False),
            nn.Sigmoid()
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        b, c, _, _ = x.size()
        y = self.avg_pool(x).view(b, c)     # (B,C)
        s = self.fc(y).view(b, c, 1, 1)     # (B,C,1,1)
        return x * s
```

参考文献

- Hu, J., Shen, L., & Sun, G. (2018). *Squeeze-and-Excitation Networks*. CVPR 2018. https://arxiv.org/pdf/1709.01507

---

2) CBAM（Convolutional Block Attention Module）

```
import torch
import torch.nn as nn

class ChannelAttention(nn.Module):
    """
    CBAM: Channel attention (AvgPool+MaxPool -> shared MLP -> Sigmoid)
    """
    def __init__(self, channels: int, reduction: int = 16):
        super().__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.max_pool = nn.AdaptiveAvgPool2d(1)
        self.mlp = nn.Sequential(
            nn.Conv2d(channels, channels // reduction, kernel_size=1, bias=False),
            nn.ReLU(inplace=True),
```

```
            nn.Conv2d(channels // reduction, channels, kernel_size=1, bias=False)
        )
        self.sigmoid = nn.Sigmoid()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        avg_out = self.mlp(self.avg_pool(x))
        max_out = self.mlp(self.max_pool(x))
        out = avg_out + max_out
        return self.sigmoid(out)

class SpatialAttention(nn.Module):
    """
    CBAM: Spatial attention (concat[AvgPool_c, MaxPool_c] -> Conv -> Sigmoid)
    """
    def __init__(self, kernel_size: int = 7):
        super().__init__()
        padding = kernel_size // 2
        self.conv = nn.Conv2d(2, 1, kernel_size, padding=padding, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # along channel dim: mean and max -> (B,1,H,W) each
        avg_out = torch.mean(x, dim=1, keepdim=True)
        max_out, _ = torch.max(x, dim=1, keepdim=True)
        x_cat = torch.cat([avg_out, max_out], dim=1)  # (B,2,H,W)
        attn = self.sigmoid(self.conv(x_cat))
        return attn

class CBAM(nn.Module):
    """
    CBAM: Channel attention -> Spatial attention
    """
    def __init__(self, channels: int, reduction: int = 16, spatial_kernel: int = 7):
        super().__init__()
        self.ca = ChannelAttention(channels, reduction)
        self.sa = SpatialAttention(spatial_kernel)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x * self.ca(x)           # channel-wise reweight
        x = x * self.sa(x)           # spatial-wise reweight
        return x
```

参考文献

- Woo, S., Park, J., Lee, J.-Y., & Kweon, I. S. (2018). *CBAM: Convolutional Block Attention Module*. ECCV 2018. https://arxiv.org/pdf/1807.06521

3) ECA（Efficient Channel Attention）

```
import math
import torch
```

```python
import torch.nn as nn

class ECABlock(nn.Module):
    """
    ECA: GAP -> Conv1d (along channel) -> Sigmoid -> channel reweight
    kernel size k is adaptively computed from C: k = |(log2(C)/gamma + b/gamma)|_odd
    """
    def __init__(self, channels: int, gamma: float = 2.0, b: float = 1.0):
        super().__init__()
        t = int(abs((math.log2(channels) / gamma) + (b / gamma)))
        k = t if t % 2 else t + 1
        k = max(k, 1)

        self.avg_pool = nn.AdaptiveAvgPool2d(1)          # (B,C,1,1)
        self.conv = nn.Conv1d(1, 1, kernel_size=k, padding=k//2, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        b, c, _, _ = x.size()
        y = self.avg_pool(x).view(b, 1, c)     # (B,1,C)
        y = self.conv(y)                        # (B,1,C)
        s = self.sigmoid(y).view(b, c, 1, 1)   # (B,C,1,1)
        return x * s
```

参考文献

- Wang, Q., Wu, B., Zhu, P., Li, P., Zuo, W., & Hu, Q. (2020). *ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks*. CVPR 2020. https://arxiv.org/pdf/1910.03151

---

4) CA（CoordAttention，Coordinate Attention）

> 思想：把全局池化拆成**横向/纵向**两条路径，编码**位置信息**并进行通道重标定；对小目标/检测友好。

```python
import torch
import torch.nn as nn

class HSwish(nn.Module):
    def forward(self, x):  # h-swish = x * relu6(x+3)/6
        return x * torch.relu6(x + 3.) / 6.

class CoordAtt(nn.Module):
    """
    Coordinate Attention:
      - Split global pooling into H and W directions
      - Fuse -> 1x1 conv (reduce) -> BN -> h-swish
      - Branch to height and width attention -> sigmoid
      - Broadcast and reweight
    Paper usually uses reduction ratio (e.g., r=32)
    """
    def __init__(self, channels: int, reduction: int = 32):
        super().__init__()
```

```
        self.pool_h = nn.AdaptiveAvgPool2d((None, 1))  # keep H, pool W
        self.pool_w = nn.AdaptiveAvgPool2d((1, None))  # keep W, pool H

        mid_ch = max(8, channels // reduction)
        self.conv1 = nn.Conv2d(channels, mid_ch, kernel_size=1, stride=1, padding=0,
bias=False)
        self.bn1   = nn.BatchNorm2d(mid_ch)
        self.act   = HSwish()

        self.conv_h = nn.Conv2d(mid_ch, channels, kernel_size=1, stride=1, padding=0,
bias=False)
        self.conv_w = nn.Conv2d(mid_ch, channels, kernel_size=1, stride=1, padding=0,
bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        b, c, h, w = x.size()
        # 方向分离的全局池化
        x_h = self.pool_h(x)                    # (B,C,H,1)
        x_w = self.pool_w(x).permute(0,1,3,2)   # (B,C,1,W) -> (B,C,W,1)

        y = torch.cat([x_h, x_w], dim=2)        # (B,C,H+W,1)
        y = self.conv1(y)
        y = self.bn1(y)
        y = self.act(y)

        # 拆回两个分支
        y_h, y_w = torch.split(y, [h, w], dim=2)  # (B,mid,H,1), (B,mid,W,1)
        y_w = y_w.permute(0,1,3,2)                # (B,mid,1,W)

        a_h = self.sigmoid(self.conv_h(y_h))      # (B,C,H,1)
        a_w = self.sigmoid(self.conv_w(y_w))      # (B,C,1,W)

        out = x * a_h * a_w
        return out
```

参考文献

- Hou, Q., Zhou, D., & Feng, J. (2021). *Coordinate Attention for Efficient Mobile Network Design*. CVPR 2021. https://openaccess.thecvf.com/content/CVPR2021/papers/Hou_Coordinate_Attention_for_Efficient_Mobile_Network_Design_CVPR_2021_paper.pdf

---

使用与放置建议（实践经验）

- **放置位置**：常见在**残差块的输出**（残差融合后）或**每个 stage 的最后一个块**；检测网络中也常见在 C3/ELAN 模块末尾。
- **开销权衡**：
  - **ECA** 最轻，移动端/实时任务优先；
  - **SE** 表达力强但参数略多；
  - **CBAM** 同时做通道+空间，效果稳妥；

    - **CA** 对检测/小目标/位置信息敏感任务常有效。
- **注意**：这些是“隐式注意力”，**没有 Q/K/V**；它们通过池化/卷积/MLP 学习加权函数来实现“关注”。